

|     |               |     |
|-----|---------------|-----|
| 000 | lFontttfamily | 050 |
| 001 |               | 051 |
| 002 |               | 052 |
| 003 |               | 053 |
| 004 |               | 054 |
| 005 |               | 055 |
| 006 |               | 056 |
| 007 |               | 057 |
| 008 |               | 058 |
| 009 |               | 059 |
| 010 |               | 060 |
| 011 |               | 061 |
| 012 |               | 062 |
| 013 |               | 063 |
| 014 |               | 064 |
| 015 |               | 065 |
| 016 |               | 066 |
| 017 |               | 067 |
| 018 |               | 068 |
| 019 |               | 069 |
| 020 |               | 070 |
| 021 |               | 071 |
| 022 |               | 072 |
| 023 |               | 073 |
| 024 |               | 074 |
| 025 |               | 075 |
| 026 |               | 076 |
| 027 |               | 077 |
| 028 |               | 078 |
| 029 |               | 079 |
| 030 |               | 080 |
| 031 |               | 081 |
| 032 |               | 082 |
| 033 |               | 083 |
| 034 |               | 084 |
| 035 |               | 085 |
| 036 |               | 086 |
| 037 |               | 087 |
| 038 |               | 088 |
| 039 |               | 089 |
| 040 |               | 090 |
| 041 |               | 091 |
| 042 |               | 092 |
| 043 |               | 093 |
| 044 |               | 094 |
| 045 |               | 095 |
| 046 |               | 096 |
| 047 |               | 097 |
| 048 |               | 098 |
| 049 |               | 099 |

# Quantify Memorization in Quantum Code Generation With Code Rewriting

Anonymous CLEAR-TEXT submission

## Abstract

added later.

**Keywords:** large language models, quantum code generation, quantum Memorization, data contamination, interpretability-driven analysis of model behaviour.

## 1 Introduction

Large language models (LLMs) have demonstrated the ability to generate quantum programs, yet their functional performance remains limited, as reflected by the relatively low accuracy achieved on existing quantum-code benchmarks. At the circuit level, quantum programs are fundamentally composed of quantum gates and their arrangement over qubit wires, while the same quantum functionality can often be implemented through multiple equivalent gate decompositions and circuit structures. Prior studies have shown that code-generation benchmarks may suffer from training-corpus contamination and that LLMs can reproduce memorised patterns. However, existing work has not examined whether such contamination and Memorization are associated with specific components of quantum-circuit knowledge, such as gate selection and circuit structure. Distinguishing which parts of quantum knowledge are primarily supported by corpus exposure or memorised representations could provide a more targeted basis for improving training data and model optimisation, thereby promoting genuine quantum-code generation capability rather than surface-level pattern reproduction.

To address this gap, we introduce a new paired benchmark in which each task pair preserves the same intended quantum semantics while modifying either the gate-level implementation or the circuit-level arrangement.

By comparing model performance across original and rewritten tasks after transpiling circuits into a common basis, we analyse whether model competence transfers across alternative implementations and whether rewrite sensitivity is concentrated at the gate or structural level. In addition, we introduce quantum-specific similarity measurements, including ( $S_{\text{quantum}}$ ), *together with common – basis transpilation as a representation – control mechanism, to separate conventional code overlap from relevant similarity and reduce gate – set – dependent measurement bias.*

This paper we propose a pipeline to measure the performance drop after code rewriting, making the following contributions: 1.A set of Quantum-aware Memorization metrics. Formal definitions of Quantum similarity score( $s_{\text{gate}}, s_{\text{structure}}, s_{\text{func}}$ ), extending classical code-LLM Memorization frameworks;

2.A Quantum-specific corpus contamination detection method that can distinguish quantum-specific implementation overlaps and executable function overlaps from a general tutorial, benefit to improve the accuracy of the materials needed for training corpora.

3.A Quantum-specific evaluation framework which can reduce benchmark representation bias through transpilation and identify a model’s mastery of specific circuit knowledge modules.

Overall, this work provides, to our knowledge, the first framework for jointly quantifying corpus contamination and Memorization in terms of underlying quantum-circuit knowledge. Rather than treating quantum-code Memorization as a single scalar phenomenon, the proposed framework distinguishes functional, gate-level, and structural responses and

provides corresponding metrics for analysing them. These results establish a new methodology for studying how training-corpus exposure relates to quantum-code generation behaviour and provide a basis for designing more targeted data curation, training, and evaluation strategies aimed at improving genuine quantum-programming capability.

## 2 Related Work

### Quantum Code Generation Benchmarks and Datasets

Qiskit HumanEval (?) adapts the HumanEval format to quantum computing, providing 101 hand-curated tasks with prompts, canonical Qiskit solutions, and unit tests across eight task categories, primarily targeting executable correctness and Qiskit API usage. QuanBench (?) extends this work with 44 Qiskit-based tasks covering quantum algorithms, state preparation, gate decomposition, and QML, evaluated via Pass@K and process fidelity. Multiple LLMs evaluated in QuanBench achieve less than 40% overall accuracy, with common failure modes including outdated API usage, circuit construction errors, and incorrect algorithm logic.

**Memorization and Data Contamination in Code LLMs** In classical code generation, Zhang et al. (?) propose an automated code-rewriting pipeline that generates semantically equivalent but structurally different problems and define the MRI, combining structural similarity (e.g., AST overlap) with the relative accuracy drop on rewritten tasks. The study finds that larger models often exhibit reduced Memorization risk, while supervised fine-tuning can substantially increase Memorization even as it improves accuracy, and that reinforcement learning with PPO yields a more balanced trade-off. Complementary contamination work includes Riddell et al. (?), who inject contamination during pre-training and fine-tuning and demonstrate non-uniform impacts on evaluation scores, challenging the assumption that contamination uniformly inflates reported performance. The structural similarity component of qMRI also connects to the code clone detection literature. Roy et al. (?) provide a foundational taxonomy of clone types (Type-1 through Type-4), rang-

ing from exact copies to semantically equivalent but syntactically dissimilar programs, and compare detection techniques based on token sequences, abstract syntax trees, and program dependency graphs. The qMRI similarity measure  $\text{sim}(\hat{c}, c^*)$  is analogous to a Type-2 to Type-3 clone score: it captures near-copy and renamed-identifier matches rather than exact replication, and inherits the well-studied precision-recall trade-offs of AST-based clone detectors.

## 3 Methodology

The methodology comprises four main components: (i) benchmark and dataset corpus curation, (ii) quantum-aware Memorization metric design, (iii) experimental evaluation across models and training regimes.

### 3.1 Contamination detection

We assess training corpus contamination through a three-stage, progressively deeper analysis pipeline. **Stage 1: Surface-Level Memorization Coverage.** Adapted from the Memorization detection framework of [MovieLens citation], in which an item is considered memorised if the model can reproduce an exact known record from a partial prompt, we define each benchmark item as a triple  $D_i = (x_i, c_i, e_i)$ , where  $x_i$  is the task prompt,  $c_i$  is the canonical Qiskit solution, and  $e_i$  is the evaluator/test oracle. We then define the *Quantum Memorization Coverage* (QMC) as:

$$QMC = \frac{|\{i : \text{Sim}(\text{predicted solution } c'_i, \text{canonical solution } c_i) > \text{acertainthreshold}\}|}{D}, \text{ similarity threshold} = 80\%.$$

QMC quantifies the fraction of benchmark solutions recoverable from the model’s parameters, providing a direct estimate of verbatim Memorization at the item level.

We define two Probe Types:

Probe A (Task-ID): Only `task_id` is provided, Model can only rely on training memory for recall, Verbatim memory.

Probe B (Prefix): Function signature + docstring is provided → Model continues writing the function body → Inference

**Stage 2: Corpus-Level Syntactic and Semantic Similarity.** To quantify the degree to which the training corpus over-

laps with benchmark solutions beyond exact string matches, we adopt the similarity scoring scheme of Dolos [citation], defining a composite programme similarity score as:

Contamination scores:  $Scorpus = \max(S_{textsurface}(p, p^*), S_{textsemantic}(p, p^*))$ , where  $S_{textsurface}$  captures token-level lexical overlap and  $S_{textsemantic}$  captures structural or embedding-based similarity. This aggregated score reflects the intuition that two programmes may be similar either in their surface form or in their underlying semantics, and taking the maximum provides a conservative upper bound on their resemblance. We further leverage the pass@ $k$  DROP metric (after removing the high *con\_score* task) to surface contamination signals: a benchmark task whose nearest corpus neighbour has a high composite similarity  $S$  is expected to exhibit a larger DROP under rewriting, since the model’s correct performance on the original prompt is more likely driven by Memorization than by genuine understanding.

**Stage 3: Spatial Autocorrelation of Contamination and Performance.** Finally, we apply Moran’s  $I$  statistic to test whether contamination scores are spatially correlated with pass@ $k$  values across the task space. If contamination drives performance, tasks that are structurally similar to one another should exhibit similar pass@ $k$  scores; a genuinely capable model, by contrast, should show no such clustering. A statistically significant result ( $p < 0.05$ ) indicates that pass@ $k$  co-varies with corpus proximity rather than being independently distributed across tasks, providing evidence that contamination influences benchmark outcomes.

### 3.2 Quantum-Aware Memorization Metrics(S\_quantum)

Quantum programmes differ fundamentally from classical code in that their correctness is defined not by deterministic output but by the probability distribution over measurement outcomes produced after simulating a parameterised sequence of gate operations on qubits. Evaluating whether a model has genuinely learned a quantum task therefore requires probing three distinct levels of the quantum execution pipeline: (i) the *type and count* of gates applied, (ii) the *structural topology* of

the resulting circuit, and (iii) the *functional output distribution* over measurement results. The process is shown in Fig.3

We correspondingly define three complementary similarity metrics.

**1.Gate Similarity (  $s\_gate$  ):**  $s\_gate$  measures how similar the gate compositions of two circuits are. It compares gate-frequency vectors using cosine similarity. For example, given gate vocabulary  $[g_1, g_2, g_3]$  with canonical histogram  $[1, 2, 3]$  and generated histogram  $[3, 4, 1]$ , we obtain  $S_{gate} = \cos([1, 2, 3], [3, 4, 1])$ .

Let  $C$  be the candidate circuit and  $C^*$  the canonical circuit. Let  $h(C)$  denote the gate-count vector of  $C$  over a shared gate vocabulary. The score is:

$$s_{\text{gate}}(C, C^*) = \frac{h(C)^\top h(C^*)}{\|h(C)\|_2 \|h(C^*)\|_2}$$

The score ranges from 0 to 1:  $0 \leq s_{gate} \leq 1$ . A higher value means that the two circuits use more similar proportions of gates. It does not mean that the gates appear in the same order or implement the same function.

**2.Structure Similarity ( $s_{struc}$ ):** Beyond gate counts, the connectivity and ordering of gates within a circuit determine its computational semantics. We represent each circuit as a gate-dependency directed acyclic graph (DAG), where nodes correspond to gate operations and directed edges encode data dependencies between gates acting on shared qubits.

$s_{\text{struct}}$  measures how well the candidate circuit covers the canonical circuit’s operation sequence on each qubit. For each canonical qubit  $q$ , define:  $S_q^* = S_q(C^*)$  as the canonical operation sequence on that qubit, and  $S_q = S_q(C)$  as the corresponding candidate sequence. Gate names are normalized before comparison, including the removal of automatically generated numeric suffixes.

**$k$ -gram coverage** The default value is  $k = 2$ . Let  $\Gamma_k(S_q^*)$  be the multiset of canonical  $k$ -grams, and let  $N_S(\gamma)$  be the number of occurrences of  $k$ -gram  $\gamma$  in sequence  $S$ .

The per-qubit  $k$ -gram coverage is:

$$K_q = \frac{\sum_{\gamma \in \Gamma_k(S_q^*)} \min N_{S_q^*}(\gamma), N_{S_q}(\gamma)}{\sum_{\gamma \in \Gamma_k(S_q^*)} N_{S_q^*}(\gamma)}$$

If the canonical sequence contains fewer than two operations, the implementation falls back to unigram coverage:  $k = 1$ . If the canonical sequence on a qubit is empty, its coverage is defined as:  $K_q = 1$ . **Longest-common-subsequence coverage**

The second component measures how much of the canonical operation order is preserved:

$$L_q = \frac{\text{LCS}(S_q^*, S_q)}{|S_q^*|}$$

If the canonical sequence is empty, the implementation defines:

$$L_q = 1.$$

#### Circuit-level aggregation

Let  $Q^*$  be the set of qubits in the canonical circuit. The two components are averaged over canonical qubits:

$$\begin{aligned} \bar{K} &= \frac{1}{|Q^*|} \sum_{q \in Q^*} K_q \\ \bar{L} &= \frac{1}{|Q^*|} \sum_{q \in Q^*} L_q \end{aligned}$$

The final structural score is:

$$s_{\text{struct}}(C, C^*) = \max(\bar{K}, \bar{L})$$

**3.Function Similarity ( $f$ )** Quantum circuits are inherently probabilistic, which precludes exact output matching as a correctness criterion. The functional score  $f$  measures whether the candidate circuit and canonical circuit implement the same behavior. The calculation depends on whether the circuits contain measurements.

**Case 1: For circuits with measurements** For circuits with measurement operations, correctness is assessed primarily using total variation (TV) distance between the measurement outcome distributions of the reference and generated circuits:

$$D_{\text{TV}}(P_C, P_{C^*}) = \frac{1}{2} \sum_x |P_C(x) - P_{C^*}(x)|,$$

bounded in  $[0, 1]$ , where  $P_C(x)$  and  $P_{C^*}(x)$  denote the measurement outcome distributions of the reference and generated circuits, respectively. Functional similarity is then:

$$f(C, C^*) = 1 - D_{\text{TV}}(P_C, P_{C^*})$$

Thus:  $0 \leq f \leq 1$ . For this calculation, measurements and classical-register semantics must be retained.

Terminal measurements must not be stripped when computing  $f$ . The circuits may be transpiled internally for simulator execution, but this execution-oriented transpilation is different from the common-basis transformation used to control the representation of  $s_{\text{gate}}$  and  $s_{\text{struct}}$ .

#### Case 2: For circuits without measurements

For measurement-free circuits,  $f$  is computed using process fidelity between the full circuit operators.

For unitary circuits  $U$  and  $U^*$ , this can be written as:

$$f(C, C^*) = \frac{|\text{Tr}[(U^*)^\dagger U]|^2}{d^2}$$

where

$d = 2^n$  for an  $n$ -qubit circuit. This definition is insensitive to global phase.

**4.Treatment of evaluation failures** Failures in computing  $f$ ,  $s_{\text{s}}$ , or  $s_{\text{g}}$  are retained as NaN and analyzed separately rather than being assigned a value of zero. NaN means that similarity could not be determined. Possible causes include syntax or runtime errors, failure to construct a circuit, unresolved parameters, unsupported circuit operations, transpilation or simulation failure, incompatible quantum or classical registers, or an undefined metric such as a zero-norm gate vector. In contrast, a score of zero is a valid observed result: for example,  $f = 0$  indicates successfully evaluated but completely different circuit behaviour, while  $s_{\text{g}} = 0$  or  $s_{\text{s}} = 0$  indicates successfully computed but no gate-composition or structural agreement. Therefore, NaN values are excluded from continuous-score aggregation, while the number and proportion of failed evaluations are reported separately.

**5.Aggregation rule for the generated code candidates** As we can see in Fig.3, for

each task we generate  $n=20$  candidate code under both the original and rewritten prompts. Let  $V_{\text{ori},i}$  and  $V_{\text{rew},j}$  denote the sets of valid executable generations, failed generations are excluded rather than assigned a score of zero. We define  $m$  as the number of executable candidates, then:

$$m_{\text{ori}} = |V_{\text{ori},i}|,$$

$$m_{\text{rew}} = |V_{\text{rew},j}|.$$

The functional scores are averaged as:

$$f_{\text{ori}} = \frac{1}{m_{\text{ori}}} \sum_{j \in V_{\text{ori},i}} f_{\text{ori},i},$$

$$f_{\text{rew}} = \frac{1}{m_{\text{rew}}} \sum_{j \in V_{\text{rew},j}} f_{\text{rew},j}.$$

The same aggregation is applied to  $s_g$  and  $s_s$ .

### 6.Strip Measurements Preprocessing

The three experiments currently use different preprocessing rules. In Experiment 1 (corpus detection), no measurement-stripping step is applied. In Experiments 2 and 3, the current `strip_measurements()` function is applied before computing both the raw and transpiled versions of  $s_g$  and  $s_s$ . However, this function removes all occurrences of `measure`, `barrier`, `reset`, and `delay`, regardless of their positions in the circuit. In contrast, measurement operations are retained when computing  $f$ : measured circuits are evaluated using  $f = 1 - D_{TV}$ , while circuits without measurements are evaluated using process fidelity.

**7.Transpile Preprocessing** The original analysis already identified this concern: an increase in raw gate similarity may partly reflect compliance with the rewrite instruction rather than a change in underlying quantum knowledge. So the common-basis transpilation is primarily used to reduce potential gate-set 'hint' bias, while also enabling standardized extraction of two-qubit interaction features (such as entangling-gate count and two-qubit depth) so that we can get a clearer acknowledgment about the genuine circuit complexity.

In experiment3, we compute two versions of  $s_{\text{gate}}$  and  $s_{\text{struct}}$ :

$$s_{\text{gate}}^{\text{raw}} = s_{\text{gate}}(C, C^*) \quad , \quad \text{and} \quad s_{\text{gate}}^{\text{tran}} = s_{\text{gate}}(\tilde{C}, \tilde{C}^*)$$

$$s_{\text{struct}}^{\text{raw}} = s_{\text{struct}}(C, C^*) \quad , \quad \text{and} \quad s_{\text{struct}}^{\text{tran}} = s_{\text{struct}}(\tilde{C}, \tilde{C}^*)$$

,where both circuits are transpiled independently:

$$\tilde{C} = T(C; \mathcal{B}, \text{optimization\_level} = 0)$$

with the common basis  $\mathcal{B} = \text{RZ, SX, X, CX}$ . Then for each canonical circuit can be represented as normalization of: N1q, N2q, D2q, Dall after transpilation. Here:

- N1q: number of single-qubit basis gates;
- N2q: number of two-qubit gates after transpilation, interpreted as the entangling-primitive count (since CX is the only two-qubit primitive in the selected)
- D2q: two-qubit depth, i.e. the critical-path depth considering only CX operations;
- Dall: total circuit depth after common-basis transpilation.

### 3.3 Paired Semantically equivalent Code Rewrite Probing

The overall pipeline of the rewrite framework is illustrated in Fig 1. We describe its key stages below. The core requirement for a valid rewrite is *semantic equivalence*: two quantum circuits are considered semantically equivalent if they realise the same unitary transformation (for circuits without terminal measurement) or produce an identical probability distribution over measurement outcomes (for circuits with measurement), regardless of differences in their internal gate sequence, or qubit connectivity. We operationalise semantic equivalence through three complementary rewrite strategies:

**A.Gate Decomposition** exploits the universality of elementary gate sets: in principle, any quantum gate can be decomposed into sequences of lower-level primitives (e.g.,  $H, T, CNOT$ ), yielding structurally distinct but functionally identical circuits.

**B. Gate Commutation / Cancellation** applies commutativity relations between compatible gate pairs to reorder or eliminate redundant operations without altering the net unitary.



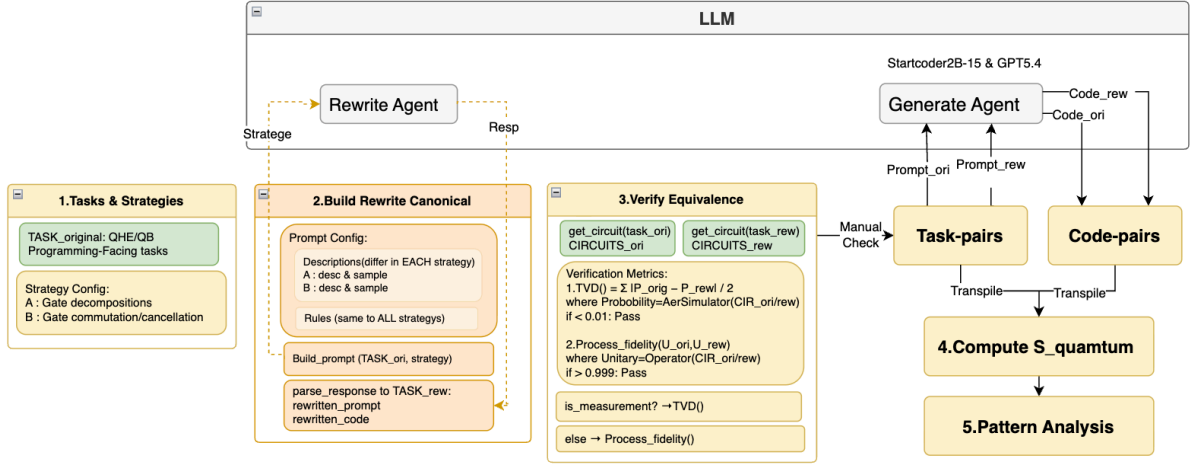


Figure 1: Quantum semantic equivalence Code Rewrite Framework

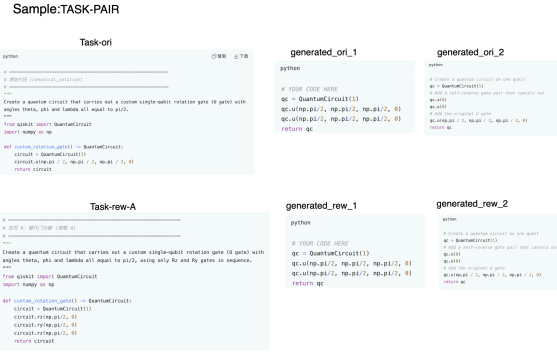


Figure 2: Sample of rewrite-pairs

To ensure that the model generates syntactically correct and semantically faithful rewrites, each rewrite strategy is specified in detail within the generation prompt alongside the original task description, so that the LLM produces both a rewritten circuit implementation and a correspondingly rephrased problem statement. All generated candidates then undergo automated equivalence verification: for measurement-based circuits, equivalence is confirmed by comparing simulated output distributions via TVD; for unitary circuits, by computing process fidelity against the canonical implementation. Only candidates that satisfy the applicable verification criterion are accepted as valid rewrites. The complete set of accepted task pairs is subsequently reviewed by domain experts to confirm that both the rewritten circuit and the rephrased problem statement are technically correct. Representative before-and-after examples are shown in Fig.2 We focus exclusively on *programmer-facing* task categories —namely algorithm

implementation, state preparation, gate decomposition, and quantum machine learning (QML) —as these require compositional reasoning about circuit structure rather than rote template completion. After processing through the pipeline described above, each benchmark task  $i$  is augmented into a *task pair*  $(x_i, x_i^r, e_i)$ , comprising the original prompt  $x_i$ , the rewritten prompt  $x_i^r$ , and a shared test oracle  $e_i$ . The test oracle is held constant across the pair because the rewrite preserves functional semantics; any divergence in pass@ $k$  between  $x_i$  and  $x_i^r$  therefore reflects a failure of the model to generalise, not a change in the evaluation criterion.

We formulate code rewriting as a paired probe of whether an LLM can transfer the same underlying quantum functionality across alternative implementations. For each benchmark task, the model is evaluated under an original prompt and a rewritten prompt that preserves the intended quantum semantics while changing the required implementation form. Conceptually, the design is analogous to testing the same underlying concept using two different problem formulations (just like when teachers design quizzes for students): a model (i.e., a student) that has acquired a transferable representation of the underlying quantum operation should remain functionally successful when the implementation constraints change, whereas a model that relies heavily on a familiar canonical pattern may become unstable under rewriting.

Two rewrite strategies were originally de-

signed to target different representation dimensions. **Strategy A** primarily modifies gate-level implementation, for example by requiring an alternative decomposition or a specified gate basis, and is therefore treated as a gate-targeted probe. **Strategy B** primarily modifies circuit ordering or organisation and was designed as a structural/wiring-targeted probe. The original experimental design explicitly aimed to use the resulting gate- and structure-level changes to distinguish these two response dimensions. Subsequent analysis, however, showed that Strategy-B rewrites containing self-inverse gate pairs are not clean order-only perturbations: inserting redundant X X, H H, or related pairs additionally changes circuit length and local gate sequences. We therefore treat **B-nonSI** as the primary structural/order probe and analyse **B-SI** separately as a mixed structural intervention. This distinction is supported empirically by the markedly different structural responses of the two B subtypes.

For each original/rewrite pair, we measure functional, gate-level, and structural responses. Functional fidelity  $f$  is the primary semantic endpoint. Gate similarity  $sg$  measures agreement in gate composition, while structural similarity  $ss$  measures agreement in the DAG-derived per-wire circuit structure. The process is shown in Fig.3 Rewrite-induced changes are defined as:

$$dp = pass@k_{rew} - pass@k_{ori}$$

$$df = f_{rew} - f_{ori}$$

$$dsg = Sg_{rew}^{tran} - Sg_{ori}^{tran}$$

$$dss = Ss_{rew}^{tran} - Ss_{ori}^{tran}$$

The superscript *tran* denotes measurement after common-basis transpilation.

How we evaluate? We introduce a three-stage evaluation framework, and all metrics phenotypes are summarized in Tab.1

### 3.3.1 Functional response phenotype

$f_{ori}$  and  $f_{rew}$  determine the functional response phenotype.

A pair that remains functionally correct under both formulations

$$f_{ori} \geq 1, f_{rew} \geq 1$$

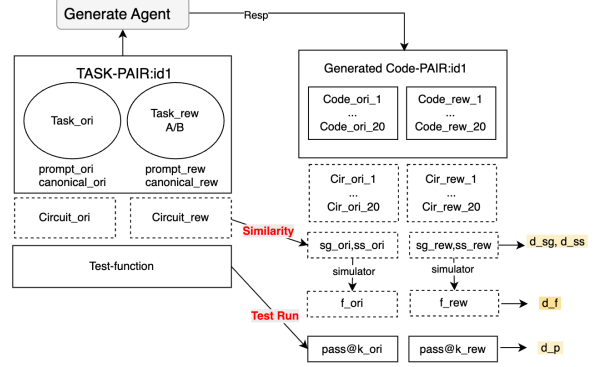


Figure 3: Metrics

is classified as **rewrite-robust functional competence**, which is consistent with generalisation across alternative implementations. This provides stronger evidence of transferable competence than success on the original formulation alone, although it does not establish unrestricted generalisation beyond the tested rewrite family.

A pair with high original fidelity but degraded rewritten fidelity

$$f_{ori} \text{ high}, f_{rew} < f_{ori}$$

is classified as **rewrite-sensitive or canonical-form-dependent behaviour**. This is the response most compatible with the original Memorization hypothesis: the model succeeds on the familiar formulation but loses functional accuracy when the same knowledge is expressed differently. We deliberately describe this as *Memorization-compatible* rather than as proof of Memorization, because rewrite failure can also arise from instruction following or implementation errors. Conversely,

$$f_{rew} > f_{ori}$$

is treated as **rewrite-assisted recovery** (rather than Memorization). In such cases, the rewritten formulation helps the model recover functionality that was absent or weak in the original condition. Finally, failure under both formulations provides **no demonstrated functional mastery**.

The continuous  $df$  remains useful as a within-task measure of rewrite sensitivity. However, it is not used directly to compare Memorization strength across models, because its magnitude depends strongly on the original performance level.



| Merics response                                   | Interpretation                                       |
|---------------------------------------------------|------------------------------------------------------|
| $\text{fori} \simeq 1, \text{frew} \simeq 1$      | Rewrite-robust competence, align with generalisation |
| $\text{fori} \simeq 1, \text{frew} < \text{fori}$ | Memorization degradation(Canonical-form dependence)  |
| $\text{fori} < \text{frew}, \text{frew} \simeq 1$ | Rewrite rescue(prompt facilitation)                  |
| $\text{fori} \simeq 0, \text{frew} \simeq 0$      | No relevant internal knowledge                       |
| $0 < \text{fori} < 1, 0 < \text{frew} < 1$        | No demonstrated functional mastery                   |
| $\text{dsg} < 0, \text{dss} \approx 0$            | Gate-sensitive canonical dependence                  |
| $\text{dsg} \approx 0, \text{dss} < 0$            | Structure-sensitive canonical dependence             |
| $\text{dsg} < 0, \text{dss} < 0$                  | mixed representation dependence                      |

Table 1: Metrics response phenotype

### 3.3.2 Circuit implement Interpretation

Only after the functional phenotype is established do  $\text{dsg}$  and  $\text{dss}$  localise how the model’s implementation response changed. A positive

$$\text{dsg} > 0$$

means that the rewritten output is more aligned with the rewritten canonical circuit at the gate-composition level than the original output was with the original canonical circuit. It therefore indicates **successful gate-level adaptation**, not additional gate memory per se. Likewise,

$$\text{dss} > 0$$

indicates improved structural agreement under the rewritten formulation. Within a rewrite-sensitive functional case, these representation-level changes can be used diagnostically. A predominantly negative  $\text{dsg}$  with relatively stable  $\text{dss}$  is consistent with a gate-sensitive implementation failure; a predominantly negative  $\text{dss}$  with relatively stable  $\text{dsg}$  indicates structure-sensitive failure; degradation in both suggests mixed representation dependence. Conversely, if  $\text{f}$  remains close to one while  $\text{sg}$  or  $\text{ss}$  changes substantially, the model has preserved semantics despite changing implementation representation, which is more consistent with transferable competence than canonical-pattern dependence.

### 3.3.3 Cross-model stability analysis

Cross-model stability requires additional adjustment because different LLMs may have substantially different baseline  $\text{fori}$  distributions. Raw  $\text{df} = \text{frew} - \text{fori}$  can therefore confound rewrite sensitivity with baseline competence and the available upward or downward

headroom. Following the baseline-adjusted ANCOVA formulation described by Vickers and Altman (2001), we model rewritten fidelity as a function of original fidelity and LLM identity:

$$\text{frew} = \beta_0 + \beta_1 \text{fori} + \beta_2 I(\text{LLM2}) + \epsilon,$$

where LLM1 is used as the reference category and cluster-robust standard errors are computed by base task ID.

To obtain directly comparable model-level estimates, both models are evaluated at the same reference value  $c$ , chosen as the grand mean of  $\text{fori}$ . Their adjusted rewritten fidelities are

$$\text{frew}^{\text{LLM1}}(c) = \beta_0 + \beta_1 c,$$

and

$$\text{frew}^{\text{LLM2}}(c) = \beta_0 + \beta_1 c + \beta_2.$$

These adjusted means answer the descriptive question: **if both models started from the same level of original functional competence, what rewritten fidelity would each model be expected to achieve?**

Because the primary model contains no model $\times$ fori interaction, subtracting the two adjusted predictions cancels the common baseline term:

$$\text{frew}^{\text{LLM2}}(c) - \text{frew}^{\text{LLM1}}(c) = \beta_2$$

Thus,  $\beta_2$  is exactly the baseline-adjusted difference in rewrite fidelity between LLM2 and LLM1. Its confidence interval and p-value provide the inferential test of whether the two models differ in adjusted rewrite stability, whereas the adjusted  $\text{frew}$  values describe the

expected rewrite performance of each model at the common baseline. We used the model coefficient and its 95% confidence interval to quantify the baseline-adjusted between-LLM difference. Because  $f$  is bounded on a 0–1 scale, the coefficient is directly interpretable as the absolute difference in rewritten functional fidelity.

## 4 Experiment

**Experiment1: Corpus Detection** The corpus-analysis stage uses the same continuous functional-similarity definition when an executable corpus circuit can be recovered. Measurement-based canonical-corpus pairs are compared using  $S_f = 1 - d_{TV}$  with 32768 shots, whereas non-measurement pairs use process fidelity.

**Experiment2: Quantum semantic equivalence task-pairs** During rewrite-benchmark construction, functional metrics are used as an acceptance criterion to verify that each canonical rewrite preserves the semantics of its original task. For measurement-based canonical equivalence checks:

$N_{\text{shots}} = 32768, \text{seed} = 42$ . The accepted TVD threshold is:

$\tau_{\text{TVD}} = \min\left(0.02, 3\sqrt{\frac{2^{n_q}}{N_{\text{shots}}}}\right)$ . A rewrite is accepted when:

$D_{\text{TV}} \leq \tau_{\text{TVD}}$ . For measurement-free circuits, the process-fidelity criterion is:

$$f \geq 1 - 10^{-6}.$$

**Experiment3: Generated-code evaluation Cross LLM**

For measurement-based tasks, model outputs are scored as  $N_{\text{shots}} = 32768, \text{seed} = 42$ . These settings serve a different purpose from canonical rewrite verification: Rewrite verification makes a binary decision about whether a transformation preserves semantics. Generated-code evaluation uses  $f$  as a continuous performance score. The two settings should therefore not be presented as interchangeable.

## 5 Analysis

Our analysis consists of three steps, with the conclusions of each step corresponding to Finding 1, 2, and 3: (i) demonstrates why classical-code similarity is insufficient; (ii) justifies our proposed quantum-specific similarity; and (iii)

reveals the behavior of each model within our evaluation framework.

**F1. Conventional code similarity overestimates quantum-specific contamination.** Our contamination analysis in Fig.4 shows that conventional code similarity captures training-corpus exposure that is systematically associated with benchmark performance, but should not be interpreted directly as quantum-task-specific contamination.

**F1.1.Item-level verbatim Memorization(direct task-ID extraction) and context-conditioned reconstruction are distinct phenomena.** Probe A provides only the QiskitHumanEval task identifier and explicitly requests the canonical solution. Under this setting, both GPT-5.4 and Grok-4 achieve 0% QMC. Thus, we observe no evidence of direct task-ID-to-solution extraction under this strongest verbatim-recall probe. In contrast, Probe B yields non-zero coverage when the function signature and docstring are supplied, demonstrating that canonical-like implementations can be reconstructed once semantic task context is available.

**F1.2.High-Scorpus tasks are performance-enriched:** targeted removal of high-overlap tasks substantially reduced  $\text{pass}@1$ , and this effect persisted under difficulty-controlled and matched random-removal baselines;  $\text{pass}@1$  also exhibited significant positive autocorrelation in the Scorpus neighbourhood space. Although conventional corpus similarity is not a direct measure of quantum-specific contamination, it contains a clear performance-associated signal. In QHE,  $\text{pass}@1$  decreases from **4.24% to 0.83%** after removing the 52% of tasks with Scorpus 60, corresponding to a **3.41-point reduction**. In QB, excluding the 32% of tasks with Scorpus 70 reduces  $\text{pass}@1$  from **6.82% to 2.67%**, a **4.15-point (60.89%) reduction**.

The effect persists within difficulty strata and is substantially larger than matched random removal. QHE basic and intermediate tasks decline by 88.71% and 50.92%, respectively, while QB basic and intermediate tasks decline by 59.38% and 52.38%. At the corresponding removal ratios, 500 random-removal trials produce only  $-0.12 \pm 1.25$  pp

|                           |                                                                                                                                                                                                         |                    |                |             |      |                 |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|----------------|-------------|------|-----------------|
| Baseline                  | QHE Full (remove <i>S_corpus</i> ) DROP, pass@1_full = 4.24<br>S>60 = 0.83 Rm = 0.52 DROP = -3.41(-80.34%)<br>S>70 = 3.72 DROP = -0.52(-12.31%) Rm = 0.25<br>S>80 = 4.46 DROP = 0.22(+5.24%) Rm = 0.08  |                    |                |             |      |                 |
|                           | QB Full (remove <i>S_corpus</i> ) DROP, pass@1_full = 6.82<br>S>60 = 1.25 DROP = -5.57(-81.67%) Rm = 0.64<br>S>70 = 2.67 Rm = 0.32 DROP = -4.15(-60.89%)<br>S>80 = 5.24 DROP = -1.58(-23.17%) Rm = 0.05 |                    |                |             |      |                 |
| Drop by Difficulty        | QHE Difficulty-bracket DROP (remove: <i>S_corpus</i> >60)<br>bracket n n_Rm(%) p_full% p_de% DROP                                                                                                       |                    |                |             |      |                 |
|                           | basic                                                                                                                                                                                                   | 79                 | 51(0.645)      | 6.33        | 0.71 | -5.61(-88.71%)  |
|                           | intermediate                                                                                                                                                                                            | 67                 | 28(0.42)       | 2.09        | 1.03 | -1.06(-50.92%)  |
|                           | difficult                                                                                                                                                                                               | 5                  | 0              | 0.00        | 0.00 | +0.00(+nan%)    |
|                           | QB Difficulty-bracket DROP (remove: <i>S_corpus</i> >70)<br>bracket n n_Rm(%) p_full% p_de% DROP                                                                                                        |                    |                |             |      |                 |
|                           | basic                                                                                                                                                                                                   | 13                 | 5(0.38)        | 18.46       | 7.50 | -10.96(-59.38%) |
|                           | intermediate                                                                                                                                                                                            | 20                 | 6(0.3)         | 3.00        | 1.43 | -1.57(-52.38%)  |
|                           | difficult                                                                                                                                                                                               | 9                  | 3              | 0.00        | 0.00 | +0.00(+nan%)    |
| randomly remove           | (remove green-align with baseline-RM ratio)<br>Pass@1 Drop_random mean±std (n=500) DROP                                                                                                                 |                    |                |             |      |                 |
|                           | QHE (Rm:0.52)                                                                                                                                                                                           | 4.24               | -0.12 ± 1.25   |             |      | (-2.73%)        |
|                           | QB (Rm:0.32)                                                                                                                                                                                            | 6.82               | +0.19 ± 1.86   |             |      | (+2.74%)        |
| Moran'I -pass@k<br>&Score | (whether pass@k is also similar when <i>s_score</i> is close,<br>p < 0.05 means it's significant.)                                                                                                      |                    |                |             |      |                 |
|                           | QHE:K=20                                                                                                                                                                                                | Global I = +0.0491 | E[I] = -0.0067 | z = +2.5857 |      | p = 0.0230      |
|                           | QB:K=10                                                                                                                                                                                                 | Global I = +0.0941 | E[I] = -0.0233 | z = +2.2766 |      | p = 0.0300      |

Figure 4: Drop after removing high  $S_{corpus}$  tasks & Moran's I

| Benchmark | Metric | Valid / total | Coverage | Mean | r with $S_{corpus}$ | p    |
|-----------|--------|---------------|----------|------|---------------------|------|
| QHE       | $S_g$  | 75/151        | 49.7%    | .480 | .346                | .002 |
| QHE       | $S_s$  | 17/151        | 11.3%    | .484 | -.300               | .242 |
| QHE       | $S_f$  | 9/151         | 6.0%     | .553 | .446                | .229 |
| QB        | $S_g$  | 39/42         | 92.9%    | .507 | .093                | .575 |
| QB        | $S_s$  | 10/42         | 23.8%    | .194 | .825                | .003 |
| QB        | $S_f$  | 8/42          | 19.0%    | .060 | .392                | .336 |

Figure 5: Spearman between  $S_{quantum}$  and  $S_{corpus}$ . Here,  $r$  is the cross-sectional Pearson correlation calculated once across all valid tasks.

for QHE and  $+0.19 \pm 1.86$  pp for QB. Consistent with the deletion analysis, pass@1 exhibits significant positive global autocorrelation over the primary corpus-similarity neighbourhood graphs:  $I=0.0491, p=.023$  for QHE and  $I=0.0941, p=.030$  for QB. Together, these results show that high-overlap tasks form a performance-enriched region of the benchmark.

However, decomposing conventional similarity into gate-, structure-, and function-level quantum diagnostics revealed that programme-level overlap does not consistently translate into quantum-semantic overlap: **F1.3. Quantum-aware diagnostics expose false positives in generic corpus overlap.** *Scorpus* is moderately associated with  $S_g$  in QHE ( $r=.346, p=.002, n=75$ ) whereas the association is absent in QB; but the strong structural correlation with  $S_s$  is in QB

( $r=.825, p=.003, n=10$ ), whereas the association is absent in QHE. Thus, conventional source similarity can sometimes track gate-composition overlap, but this relationship is benchmark-dependent rather than universal. Moreover, the further Quantum-aware review shows that many moderate- or high-Scorpus candidates corresponded to generic Qiskit/pytket backend tests or repeatedly retrieved framework snippets rather than implementations of the target quantum task (i.e, gate-, structure-, or function-level quantum similarity). Through the current final retrieval results where we have saved the file name and snippets when computing the *Scorpus*, we find that: a) **23/151 QHE tasks (15.2%)** map to the same pytket-qiskit/tests/backend\_test.py, while **15/42 QB tasks (35.7%)** map to either backend\_test.py or qiskit\_backend\_test.py; b) Analogously, there is obvious candidate reuse among these 10 structurally comparable samples which related to the apparently strong QB structural correlation: 4 come from /InjectiveGrover.py, and another 2 come from the same RemoveBarriers.py. **[ShowCase placeholder : shared files, 1-to-many]**

**F1.4. Sf 1 provides stronger circuit-semantic contamination evidence.**

**[ShowCase placeholder: snippet and corresponding code, 1-to-1, QHE89 and QHE92]**

Because  $S_f$  requires both the retrieved corpus code and the canonical implementation to be reconstructed as executable and functionally comparable quantum circuits, its coverage is substantially lower than that of source-, gate-, or structure-level metrics. We therefore use  $S_f$  as high-confidence case-level semantic evidence rather than as a population-level trend statistic. The  $S_f 1$  cases expose forms of corpus overlap that cannot be reliably identified from representation-level metrics alone. In QHE89, both the canonical task and the retrieved corpus candidate construct the same multi-controlled Hadamard operation using `HGate().control(2)`; despite  $S_s=0$ ,  $S_f 1$ , showing that structural mismatch does not preclude functional equivalence. In QHE92, the corpus candidate reproduces the Bell-state preparation, `StabilizerState` con-

struction, and `probabilities_dict()` operation of the benchmark task, while  $S_g$ ,  $S_s$ , and  $S_f$  are all approximately 1. These cases demonstrate that functional similarity provides substantially more direct **circuit-semantic exposure evidence** than generic programme overlap, gate histograms, or per-wire structural agreement alone.

Overall, we therefore interpret *Scorpus* as a **performance-associated corpus-exposure signal**, while using  $S_g$ ,  $S_s$ , and  $S_f$  as progressively more quantum-specific diagnostic evidence.

**F2.Quantum semantics is not uniquely coupled to circuit representation: Structural similarity normalized by common-basis transpilation should be interpreted in conjunction with functional correctness.** We first perform a measurement sanity check on canonical original/rewrite pairs, and then observe the actual performance of the model-generated code to determine when these metrics are reliable and when they cannot replace semantic correctness.

**F2.1.Semantic equivalence does not imply a unique  $S_g$  or  $S_s$ .** From Fig.6 we observe that common-basis transpilation increases mean canonical-pair  $S_g$  from .526 to .963 and mean  $S_s$  from .615 to .876. However, normalization does not produce a unique representation: only 71/114 valid pairs achieve  $S_g=1$ , and only 70/114 achieve  $S_s=1$ , leaving other semantics-preserving pairs with residual representation differences. Consequently, no universal threshold on  $S_g$  or  $S_s$  alone can serve as a general test of semantic equivalence. Common-basis transpilation should therefore be interpreted as a measurement control that reduces gate-set confounding, not as semantic canonicalisation.

**F2.2.Common-basis transpilation substantially reduces measurement bias arising from gate representations** After transpiling the generated raw code then recompute circuit similarity metrics, we find that common-basis transpilation strongly compresses gate similarity toward the upper bound: for example in Fig.7, StarCoder  $S_{g,ori}$  increases from .745 to .917 and GPT-5.4 from .892 to .973, whereas  $S_s$  retains substantially more residual variation.

After transpiration I conduct the EDA in task-pairs. I computed the  $s\_gate$  and  $s\_struct$  between the canonical\_ori and canonical\_rew, but the result shows that even for the semantically equivalent pairs,  $s\_gate\_tran$  and  $s\_struct\_tran$  remain various (which I expected to be 1).

Point 1 – canonical ori vs rew similarity (after transpile)

Semantically equivalent, but structurally NOT always identical

---

n pairs (both exec OK) : 114

sg\_canano\_tran == 1 : 71/114 (62%)

ss\_canano\_tran == 1 : 70/114 (61%)

sg\_canano\_tran mean=0.963 median=1.000 min=0.000

ss\_canano\_tran mean=0.876 median=1.000 min=0.000

---

sg=1 but ss<1 (same gate mix, different wiring): 3

qiskitHumanEval/23-B sg=1.0000 ss=0.7778 N2q=0/0 Dall=1/3

01-B sg=1.0000 ss=0.9481 N2q=2/2 Dall=27/27

28-A sg=0.9999 ss=0.9667 N2q=18/18 Dall=38/39

---

Point 2 – ori vs rew complexity gap (|ΔDall| largest first)

Large gap → LLM targets two structurally different circuits → noisier signal

---

| rewrite_id            | st | N2q     | o/r     | Dall | o/r  | ΔN2q  | ΔDall | sg |
|-----------------------|----|---------|---------|------|------|-------|-------|----|
| qiskitHumanEval/147-A | A  | 54/142  | 166/376 | +88  | +210 | 0.995 |       |    |
| 25-A                  | A  | 121/135 | 450/639 | +14  | +189 | 0.999 |       |    |
| qiskitHumanEval/36-A  | A  | 4/4     | 4/52    | +0   | +48  | 0.111 |       |    |
| qiskitHumanEval/9-A   | A  | 2/2     | 14/32   | +0   | +18  | 1.000 |       |    |

Figure 6: Canonical task-pairs circuit similarity after transpilation

| Generated-code means: |             |      |              |
|-----------------------|-------------|------|--------------|
| Model                 | Metric      | Raw  | Common basis |
| StarCoder             | $s_{g,ori}$ | .745 | .917         |
| StarCoder             | $s_{g,rew}$ | .762 | .954         |
| GPT-5.4               | $s_{g,ori}$ | .892 | .973         |
| GPT-5.4               | $s_{g,rew}$ | .914 | .983         |
| StarCoder             | $s_{s,ori}$ | .667 | .747         |
| StarCoder             | $s_{s,rew}$ | .574 | .710         |
| GPT-5.4               | $s_{s,ori}$ | .845 | .913         |
| GPT-5.4               | $s_{s,rew}$ | .814 | .873         |

Figure 7: Generated code circuit similarity after transpilation

QB30-A :

$$s_{g,ori}^{raw} = 0.1565, \quad s_{g,rew}^{raw} = 0.6801$$

$$s_{g,ori}^{tran} = 0.9592, \quad s_{g,rew}^{tran} = 0.7444.$$

|            | original | rewrite |
|------------|----------|---------|
| raw        | 0.1565   | 0.6801  |
| transpiled | 0.9592   | 0.7444  |

$$d_{sg}^{raw} = 0.6801 - 0.1565 = +0.5236,$$

$$d_{sg}^{tran} = 0.7444 - 0.9592 = -0.2148.$$

Figure 8: Showcase-Starcoder QB30-A, the change direction of  $S_{gate}$  switch from positive to negative after transpilation

More importantly, we find changing only the measurement representation from raw to transpile changes (with the generated programs themselves held fixed) might reverse the inferred direction of the rewrite effect. Here if we define  $dx = Sx^{rew} - Sx^{ori}$ , where  $Sx = S_{gate}, S_{structure}$ , which indicates the performance change in the rewrite task. When  $dx$  is positive, it means that the gate/structure similarity of the rewriting task is better. A direct example of representation-sensitive gate-level inference is GPT-5.4 on QB30-A (Fig.8). For the same pair of original and rewritten model outputs, gate similarity in the raw representation increases in  $d_{sg}^{raw} = +0.5236$ . After the generated circuits and their corresponding canonical references are independently transpiled into the same common basis, however, gate similarity decreases to  $d_{sg}^{tran} = -0.2148$ , which shows the change direction of  $S_{gate}$  reverse from positive to negative. Then we check among all tasks, the  $(d_{sg}, d_{ss})$ -based directional category has been changed in the amount of 26/78 StarCoder cases (33.3%) and 12/82 GPT-5.4 cases (14.6%) .

Therefore, we draw a conclusion that raw circuit patterns partially reflect the model’s adherence to the prompt instruction, rather than a true understanding of the circuit principles. Common-basis transpilation reduces this gate-vocabulary confound, while  $S_s$ , which retains greater dynamic range than the increasingly saturated  $S_g$ , provides a more discriminative diagnostic of residual structural varia-

Common-basis  $s_s \leftrightarrow f$

Spearman

| Model     | Comparison                      | n  | $\rho$ |
|-----------|---------------------------------|----|--------|
| StarCoder | $f_{ori}$ vs $s_{s,ori}^{tran}$ | 78 | .460   |
| StarCoder | $f_{rew}$ vs $s_{s,rew}^{tran}$ | 77 | .548   |
| StarCoder | $d_f$ vs $d_{ss}^{tran}$        | 77 | .454   |
| GPT-5.4   | $f_{ori}$ vs $s_{s,ori}^{tran}$ | 78 | .513   |
| GPT-5.4   | $f_{rew}$ vs $s_{s,rew}^{tran}$ | 79 | .572   |
| GPT-5.4   | $d_f$ vs $d_{ss}^{tran}$        | 77 | .316   |

Figure 9: Sstruct’s Spearman after transpile

**【ShowCase placeholder : QB42-A, QB44-B】**

Figure 10: Enter Caption

tion.

**F2.3.Structural similarity is informative but not definitive of quantum semantics.** In the common-basis representation,  $S_s$  is significantly positively associated with functional fidelity for both StarCoder and GPT-5.4. From Fig.9 we can see for StarCoder, the Spearman correlations for the original, rewritten, and rewrite-change comparisons are .460, .548, and .454, respectively; the corresponding GPT-5.4 values are .513, .572, and .316. However, this relationship is not deterministic:  $S_s = 1$  is neither sufficient nor necessary for  $S_f = 1$ . We observe both structurally matched but functionally incorrect programs and functionally correct programs with substantial structural disagreement, which is shown in Fig.10: A case from StarCoder QB42-A, the generated program achieves both  $sg=1$  and  $ss=1$  in the common-basis representation, while its functional fidelity is  $f=0$ . This extreme counterexample demonstrates that even perfect agreement in gate composition and the measured circuit structure does not guarantee correct quantum functionality. On the other hand, the converse Case is illustrated by GPT-5.4 on decomposition task QB44-B. The rewritten program achieves perfect functional fidelity ( $f=1$ ), while its common-basis structural similarity is only  $ss=.529$  and  $sg=.944$ , which shows a functionally correct quantum program can therefore remain substantially different from the canonical implementation in structure. Taken together, structural similarity should therefore be treated as agreement with



| group            | d_f    | d_sg_tran | d_ss_tran | n (%)      | verdict |
|------------------|--------|-----------|-----------|------------|---------|
| [starcoder]      | -0.018 | 0.036     | -0.044    | 104 (100%) | sg+ ss- |
| strategy=A       | -0.02  | 0.038     | -0.026    | 66 (63%)   | sg+ ss- |
| strategy=B       | -0.014 | 0.032     | -0.072    | 38 (37%)   | sg+ ss- |
| B (self-inverse) | -0.055 | 0.016     | -0.156    | 18 (17%)   | sg+ ss- |
| B (non-SI)       | 0.021  | 0.052     | 0.037     | 20 (19%)   | -       |
| [gpt54]          | -0.081 | 0.01      | -0.043    | 104 (100%) | sg+ ss- |
| strategy=A       | -0.091 | 0.019     | -0.022    | 66 (63%)   | sg+ ss- |
| strategy=B       | -0.064 | -0.005    | -0.076    | 38 (37%)   | -       |
| B (self-inverse) | -0.153 | -0.01     | -0.135    | 18 (17%)   | -       |
| B (non-SI)       | 0.008  | 0.001     | 0.001     | 20 (19%)   | -       |

Figure 11: Dissociation in gate-level and structure-level

a reference implementation and as an auxiliary semantic diagnostic, not as independent evidence of correctness or Memorization.

**F3: Cross Model behaviour under semantics-preserving rewrites.** Semantics-preserving rewrites reveal whether model competence transfers across alternative implementations, while gate- and structural-change metrics identify the representation layer at which this transfer succeeds or fails. The resulting behaviour cannot be reduced to a single Memorization axis. Gate-level agreement and structural agreement respond differently to rewriting, these responses depend on the intervention type, and functional rewrite sensitivity is not uniform across models or task categories.

**F3.1. Gate-level and structural responses are dissociable and intervention-dependent** Let’s see the result in Fig.11. The first result partially supports the original gate-versus-wiring hypothesis, but with a more constrained interpretation. Under Strategy A, both models show the same **aggregate directional tendency**: gate agreement increases while structural agreement decreases. For StarCoder, the paired common-basis means are  $d_{sg}=+0.038, d_{ss}=-0.026$ , while GPT-5.4 shows  $d_{sg}=+0.019, d_{ss}=-0.022$ . Thus, a gate-targeted rewrite can improve agreement in which quantum primitives are used without simultaneously improving *how those operations are arranged in the circuit*. This supports the distinction between gate-level and structural response dimensions.

However, this result must be interpreted at the group level. Only 5/49 valid StarCoder Strategy-A pairs and 2/51 GPT-5.4 pairs individually occupy the strict  $d_{sg}>0, d_{ss}<0$  quadrant. Therefore, **sg+/ss-** should not be de-

| Quantity    | paired n | GPT5.4-Starcoder $\rho$ | $p$                   | Interpretation                                 |
|-------------|----------|-------------------------|-----------------------|------------------------------------------------|
| $f_{ori}$   | 84       | .416                    | $8.3 \times 10^{-5}$  | shared baseline difficulty                     |
| $f_{rew}$   | 85       | .538                    | $1.1 \times 10^{-7}$  | shared rewritten-task difficulty               |
| $d_f$       | 83       | .130                    | .242                  | no clear shared functional rewrite sensitivity |
| $s_{s,ori}$ | 75       | .518                    | $1.9 \times 10^{-5}$  | structural level partly shared                 |
| $s_{s,rew}$ | 74       | .640                    | $8.2 \times 10^{-10}$ | structural level partly shared                 |
| $d_{ss}$    | 74       | .350                    | .002                  | some shared structural response                |
| $d_{sg}$    | 72       | .016                    | .891                  | essentially no shared gate-change response     |

Figure 12: Task difficulty Spearman

scribed as the dominant individual response class. It is an **aggregate directional dissociation**, not a universal per-task pattern. This is an important refinement of the original hypothesis.

Strategy B reveals an even stronger intervention effect. Self-inverse Strategy-B is not a clean wiring probe because the inserted redundant gates themselves alter local circuit structure. Self-inverse Strategy-B rewrites produce substantial structural degradation:

StarCoder B-SI: $d_{ss}=-0.1563$ , GPT-5.4 B-SI: $d_{ss}=-0.1351$ .

In contrast, non-self-inverse Strategy-B rewrites produce approximately non-negative mean structural changes: StarCoder B-nonSI: $d_{ss}=+0.0374$ , GPT-5.4 B-nonSI: $d_{ss}=+0.0007$ .

The key finding is therefore not an internal interference mechanism, but that **structural response depends strongly on the precise form of the rewrite intervention**.

Taken together, these results revise the original “gate memory versus wiring memory” hypothesis into a measurement claim: Gate composition and circuit structure constitute dissociable response dimensions. Strategy A preferentially perturbs the gate-level representation, whereas B-nonSI provides a cleaner structural/order probe; B-SI constitutes a mixed intervention and must be analysed separately.

**F3.2. Models agree more on task difficulty than on rewrite sensitivity.** We first matched GPT-5.4 and StarCoder results by `rewrite_id`, and then computed Spearman rank correlations between the two models for  $f_{ori}$ ,  $f_{rew}$ , and  $d_f=f_{rew}-f_{ori}$ . Higher correlations for  $f_{ori}$  and  $f_{rew}$  indicate shared task-difficulty rankings, whereas the weak and non-significant correlation in  $d_f$  indicates that



| Model           | n  | raw mean $f_{ori}$  | raw mean $d_f$ | adjusted $d_f$  | 95% CI         |
|-----------------|----|---------------------|----------------|-----------------|----------------|
| StarCoder       | 87 | .512                | -.019          | -.061           | [-.120, -.002] |
| GPT             | 84 | .814                | -.081          | -.038           | [-.086, +.011] |
| Contrast        |    | Adjusted difference |                | 95% CI          | p              |
| GPT - StarCoder |    | +0.0235             |                | [-0.044, 0.091] | .496           |

Figure 13: cross-LLM adjusted

rewrite sensitivity is largely model-specific. The result is shown in Fig.12.

Across matched conditions in the primary population, original fidelity is significantly correlated between the two models ( $r=.416, p<10^{-4}$ ), and the correlation increases for rewritten fidelity ( $r=.538, p<10^{-6}$ ). This indicates a shared component of task difficulty. In contrast, rewrite-induced functional change is not significantly correlated across models ( $r=.130, p=.242$ ): a task that is strongly destabilised by rewriting for GPT-5.4 is not necessarily destabilised for StarCoder. Structural change retains a modest cross-model association ( $dss=.350, p=.002$ ), whereas gate-level change shows essentially no cross-model consistency ( $dsg=.016, p=.891$ ). Therefore StarCoder and GPT-5.4 show partial agreement on which tasks are intrinsically easy or difficult, but little agreement on which tasks are functionally affected by rewriting.

### F3.3.Baseline-adjusted functional stability does not differ overall across models

Raw comparison of  $df=f_{rew}-f_{ori}$  is strongly confounded by baseline performance because GPT-5.4 begins at substantially higher original fidelity than StarCoder (.814 vs .512), shown in Fig.13. We therefore fit an ANCOVA model with  $f_{rew}$  as the outcome and  $f_{ori}$  as the covariate, evaluating both models at the grand mean of original fidelity ( $f_{ori}=0.660$ ) to place both models at a common baseline performance level. This standardisation places the two models at the same baseline competence level and provides adjusted rewrite fidelities that can be compared independently of their different starting performance.

We then used the model coefficient for LLM identity to test whether the difference between these adjusted outcomes was statistically distinguishable from zero. After controlling for  $f_{ori}$ , the estimated GPT-StarCoder difference in rewrite fidelity was +0.0235 (95%

CI  $[-0.044, 0.091]$ ,  $p=.496$ ), providing no statistically significant evidence of an overall difference in baseline-adjusted rewrite stability.

## 6 Future work

Future work can extend the framework in four directions. First, the rewrite pairs can be used for fine-tuning rather than evaluation alone. Training on **semantically equivalent but structurally different programs**, together with **structurally similar but semantically different programs**, may reduce reliance on canonical implementation patterns and encourage stronger semantic generalisation.

Second, the rewrite taxonomy can be expanded beyond gate decomposition and circuit structure to other quantum-program principles, such as **parameterised circuits, measurement placement, ancilla usage, and qubit permutation**. This would allow Memorization and rewrite sensitivity to be analysed across a broader range of quantum knowledge.

Third, the benchmark can be extended from Qiskit to other SDKs such as **PennyLane and Cirq**. Cross-SDK evaluation would help distinguish transferable quantum knowledge from framework-specific API familiarity.

Finally, corpus-contamination analysis can be improved through finer-grained retrieval. Moving from whole-file matching to **function- or code-block-level chunks**, together with dedicated **quantum API/signature retrieval**, could reduce generic-code false positives and more precisely identify which quantum concepts are over- or under-represented in the training corpus.